0/1 背包动态规划算法讲解与代码演示

算法名称：0/1 背包动态规划（0/1 Knapsack DP）　　算法类别：动态规划 / 组合优化

讲解对象：本科生、Python 初学者　　讲解时长：35–45 分钟

基本信息

项目	填写内容
算法全称与简称	0/1 背包问题的动态规划算法；0/1 Knapsack Problem / Dynamic Programming（DP）。
算法版本与来源	经典整数容量动态规划实现；使用二维 DP、选取方案回溯及一维空间压缩。以 Bellman 动态规划思想为理论基础。
学习目标	讲清最优子结构；手工完成一个 4 物品案例；运行源代码；比较 DP 与价值密度贪心；理解空间优化与边界条件。
前置知识与环境	整数与数组、最大值函数、基本 Python 语法；Python 3.10+；算法本身仅使用标准库；图表可选依赖 matplotlib。

应用场景与问题定义

项目	填写内容
应用领域与目标任务	应急物资装载：在车辆或背包容量有限的情况下，选择若干不可拆分物资，使总效用值最高。
任务背景	物资具有不同重量和效用；按效用/重量比例排序的贪心法未必得到全局最优。动态规划适合小到中等整数容量场景。
输入数据	6 类模拟物资：简易药包(2kg,3 分)、饮用水(3kg,4 分)、应急电池(4kg,5 分)、急救箱(5kg,8 分)、照明设备(6kg,9 分)、通信设备(7kg,10 分)；容量 C=12kg。
目标输出	最优效用、物品组合、总重量、二维 DP 表、容量变化的对比结果；导出 CSV、JSON 与对比图。
任务目标与约束	最大化 $\Sigma v_i x_i$；满足 $\Sigma w_i x_i \leqslant C$、$x_i \in \{0,1\}$。每件物品最多选一次；重量与容量均为非负整数，物品重量为正。

算法直观理解

项目	填写内容
核心思想	把“大背包”分成“前 i 件物品、容量 c”的小问题：不拿第 i 件或拿第 i 件，比较两种结果并保留更优值。
微型案例	4 件物品 A(2,3)、B(3,4)、C(4,5)、D(5,8)，括号内依次为重量与效用，容量 8。最优选择 B+D，重量 3+5=8，效用 4+8=12。
执行逻辑	建立 dp[0][0..C]=0 → 按物品逐行处理 → 对每个容量计算“不选/选”的最大值 → 从末行回溯所选物品。
关键判断	若 $c<w_i$，不能选第 i 件；否则比较 dp[i−1][c] 与 dp[i−1][c−w_i]+v_i；严格更大才选择，便于稳定回溯。

核心原理与数学表达

项目	填写内容
理论依据	最优子结构：最优方案对最后一件物品必定“选”或“不选”；无后效性：前 i 件的最优值仅依赖前 i−1 件的状态。
核心公式与目标函数	$V^*=\max \Sigma_i v_i x_i$，约束 $\Sigma_i w_i x_i \leqslant C$，$x_i \in \{0,1\}$；$dp[i][c]=dp[i-1][c]$（$c<w_i$）；否则 $dp[i][c]=\max\{dp[i-1][c], dp[i-1][c-w_i]+v_i\}$；初始 $dp[0][c]=0$。
状态更新或决策规则	二维 DP 依次处理 i=1..n、c=0..C；取物品用上一行 dp[i−1]，保证不会重复选取；回溯时若采取决策则容量减去其重量。
理论性质	在整数容量及所定义约束下，DP 通过枚举全部可达子状态得到全局最优；时间 O(nC)，属伪多项式时间，不宜用于极大容量值。

符号	含义及单位	参与计算的作用
dp[i][c]	前 i 件物品、剩余可用容量上限 c 的最大效用（分）	动态规划状态；最终答案为 dp[n][C]。
w_i / v_i	第 i 件物品重量（kg）/ 效用（分）	确定能否放入与放入后增加的价值。
n / C / x_i	物品数 / 容量（kg）/ 是否选择第 i 件	限定状态规模；x_i 为 0 或 1，约束每件只选一次。

算法执行流程

阶段	执行内容	输入与输出 / 判据
数据准备	定义 6 件模拟物品及整数容量列表。	每件包含名称、重量(kg)、效用(分)。
初始化	建立 (n+1)×(C+1) 零矩阵及选择标记矩阵。	dp[0][c]=0；容量 0 的值为 0。
核心计算	逐件枚举容量：比较“跳过”和“选入”的效用。	dp[i][c] ← max(跳过, 选入)。
状态更新	保存更优值，记录是否选入当前物品。	仅从上一行读取状态，避免重复选择。
终止判断	全部物品处理完成；无需迭代收敛阈值。	i=n 时返回 dp[n][C]。
结果输出	由选择矩阵回溯；统计重量、效用并输出图表。	得到物资组合、结果 CSV/JSON 与可视化。

伪代码：

```
输入：物品列表 items=[(w₁,v₁),…,(wₙ,vₙ)]，整数容量 C
初始化 dp[0..n][0..C]=0、take[0..n][0..C]=False
对于 i=1..n：
  对于 c=0..C：
    dp[i][c]=dp[i-1][c]          # 不选第 i 件
    若 c≥wᵢ 且 dp[i-1][c-wᵢ]+vᵢ > dp[i][c]：
      dp[i][c]=dp[i-1][c-wᵢ]+vᵢ  # 选第 i 件
      take[i][c]=True
从 (i=n,c=C) 逆序回溯所有 take=True 的物品
返回 dp[n][C] 与被选中物品清单
```

参数及超参数

参数 / 符号	含义与用途	范围 / 默认值	设置依据与变化影响
n	物品数	默认 6；微型案例 4	n 增大时 DP 状态数按 n 线性增长；穷举按 2^n 增长。
C	背包最大容量	默认 12 kg；比较 8/10/12/14/16	容量是整数；C 越大，时间与状态存储量越大。
w_i	单件物品重量	2、3、4、5、6、7 kg	均为正整数；更新时用于判断容量是否足够。
v_i	单件物品效用	3、4、5、8、9、10 分	非负；更高的效用促进物品入选，但受容量约束。

代码实现结构

项目	填写内容
开发环境	Windows/macOS/Linux；Python 3.10+；标准库 dataclasses、csv、json、time；绘图可选 matplotlib 3.7+；普通 CPU 即可。
项目结构	knapsack_demo.py 为算法主程序；results/ 包含 comparison.csv、dp_table_micro.csv、dp_table_capacity12.csv、summary.json、comparison.png。
运行入口	在项目目录执行 python knapsack_demo.py；主函数 main()；输出目录基于脚本所在路径自动创建。
数据预处理	直接读取程序中的模拟物品列表；验证容量非负、重量正整数、效用非负；本例不需要特征标准化。
参数初始化	MICRO_ITEMS 为前四件物品；ITEMS 为六件物品；容量对比 [8,10,12,14,16]。
核心函数	dp_2d(items,capacity) 返回最优效用、选中索引与 DP 表；dp_1d() 使用逆序容量更新；greedy_ratio() 实现贪心对照；exhaustive() 用于正确性校验。
终止与结果返回	遍历完 n 件物品后输出结果；负容量或异常重量/效用抛出 ValueError；使用 assert 验证 2D DP、1D DP、穷举一致。
评价与可视化	最优效用（分）、贪心结果（分）、最优差额、所选总重量；输出对比 CSV、DP 表格和容量变化曲线。

代码位置	算法含义	关键输入 / 输出
dp_2d()	二维状态转移与回溯，直接对应 dp[i][c] 公式。	输入物品和 C；输出最优值、物品索引及 DP 表。
dp_1d()	容量逆序更新，空间由 O(nC) 压缩为 O(C)。	输入物品和 C；输出最优值。
greedy_ratio() / exhaustive()	实现非最优基准和小规模精确校验。	输入同一物品集合；输出对照值与物品组合。

代码关键片段（完整可运行代码见 knapsack_demo.py）：

```
for i, item in enumerate(items, 1):
    for c in range(capacity + 1):
        dp[i][c] = dp[i - 1][c]
        if c >= item.weight:
            dp[i][c] = max(dp[i][c],
              dp[i - 1][c - item.weight] + item.value)
```

现场运行演示

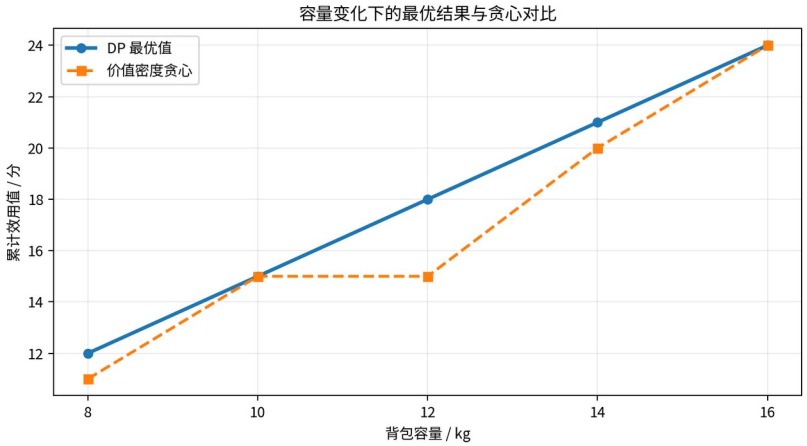
项目	填写内容
演示数据与配置	演示一：4 件物品、容量 8kg；演示二：6 件模拟物品、容量 12kg；最后比较 C=8/10/12/14/16。无随机种子（算法确定性）。
演示路径	展示物品与重量 → 公式推导 → 手工更新 DP 表 → 根据决策矩阵回溯 → 执行 Python → 对比贪心和穷举。
重点观测变量	dp[i][c]、取或不取的候选效用、最终最优值、所选索引与重量、不同容量下贪心与 DP 的差距。
运行命令与预期输出	python knapsack_demo.py；容量 12 时最优效用=18、选择“急救箱+通信设备”；comparison.csv 和 summary.json 自动生成。

运行阶段	关键变量 / 指标	状态变化或解释
初始状态	n=6、C=12；dp 的 0 行与 0 列全为 0。	还未处理任何物品，最优可用效用为 0。
中间状态	微型案例：A(2,3)、B(3,4)、C(4,5)、D(5,8)，容量 8。	处理完 D 后 dp[4][8]=max(dp[3][8]=9,dp[3][3]+8=12)=12。
结束状态	微型案例最优=12（B+D）；正式演示最优=18（急救箱+通信设备）。	两种组合均恰好装满容量；程序输出的结果与穷举枚举相同。

实验验证与结果分析

项目	填写内容
数据与实验设置	6 件模拟物品，重量 2–7kg，效用 3–10 分；分别设 C=8/10/12/14/16；程序无随机性，结果可重复。
对比方案	二维 DP（标准）、一维 DP（空间优化）、价值密度贪心（近似）、穷举（验证正确性）。
评价指标	最优效用 V*（分，越高越好）；贪心差额 Δ=V*−Vgreedy（分）；二维 DP 状态数 n(C+1)，穷举组合数 2^n。
重复与统计策略	确定性算法：相同输入运行结果一致；5 个容量条件分别比对 DP、1D DP 和穷举；本次不报告统计显著性。

实验方案	参数配置	核心指标	耗时 / 资源	结果解释
C=8 kg	二维 DP / 一维 DP / 贪心	最优 12 分；贪心 11 分	DP 状态 54；穷举 64	差额 1 分；贪心失效
C=12 kg	二维 DP / 一维 DP / 贪心	最优 18 分；贪心 15 分	DP 状态 78；穷举 64	差额 3 分；最明显
C=16 kg	二维 DP / 一维 DP / 贪心	最优 24 分；贪心 24 分	DP 状态 102；穷举 64	差额 0 分；结果一致



分析结论：本次模拟数据中，容量为 12kg 时二维 DP 与一维 DP、穷举均得到最优值 18 分（急救箱+通信设备），贪心法为 15 分，相差 3 分。五组容量试验中的精确算法输出一致；所述结论仅对本示例成立，不代表真实应急方案。

复杂度、适用性与局限

项目	填写内容
时间与空间复杂度	二维 DP：时间 O(nC)、空间 O(nC)；一维 DP：时间 O(nC)、空间 O(C)；穷举 $O(2^n)$。其中 C 是整数容量值，故称伪多项式复杂度。
适用条件与优势	物品不可拆分、独立效用可加、重量与容量能合理表示为整数；能给出精确最优方案并容易回溯解释。
关键限制与风险	容量数值过大时 DP 内存和耗时显著增加；真实场景的效用权重主观；可能出现多个同样最优的物品组合。
常见运行问题	负容量 → 抛出异常；重量为非正 → 检查输入；把 1D 容量从小到大循环会重复使用同一件物品 → 必须逆序更新。
改进与扩展方向	整数比例缩放、滚动数组、稀疏状态；多约束背包可加额外容量维度；物品可重复则改为完全背包模型。

听众检验与复现资料

检验任务	完成标准
原理复述	说明“选/不选”的两种情况、状态转移公式与基础状态。
微型案例手工推演	4 件物品、容量 8，正确推出 B+D，总重量 8kg、效用 12。
独立运行	使用 python knapsack_demo.py 生成 comparison.csv、两份 DP 表及 summary.json；可选 matplotlib 绘图。
调整参数并分析	将 C 从 8 改到 12，观察 DP 由 12 提升至 18，而贪心为 15；解释差距。

项目	填写内容
复现材料	knapsack_demo.py、README_运行说明.md、results/ 中实验数据与 PNG、课堂讲解视频；安装 Python 3.10+，直接运行；绘图需要 pip install matplotlib。
参考文献与资料	Bellman, R. (1957). Dynamic Programming. Princeton University Press；Cormen et al. (2022). Introduction to Algorithms, 4th ed., MIT Press；课程示例数据为本次自建模拟数据。